

Java Streams — 100 Scenario-Based Interview Questions

Lambda Expressions • Functional Interfaces • Collectors • Optional • Parallel Streams • Production Scenarios

Interview level: Senior Java / Full-Stack (8–15+ years). Every question includes a scenario, solution approach, and Java code snippet. Code primarily uses **Stream.toList()**; for Java 8, use **collect(Collectors.toList())**.

1. Find all even numbers

Scenario: Given a list of integers, return only the even numbers.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> result = nums.stream().filter(n -> n % 2 == 0).toList();
```

2. Find all numbers greater than 50

Scenario: Filter transactions/values greater than 50.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> result = nums.stream().filter(n -> n > 50).toList();
```

3. Square every number

Scenario: Transform each number into its square.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> result = nums.stream().map(n -> n * n).toList();
```

4. Convert names to uppercase

Scenario: Normalize customer names to uppercase.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> result = names.stream().map(String::toUpperCase).toList();
```

5. Remove duplicates

Scenario: Return unique IDs while preserving encounter order.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> result = ids.stream().distinct().toList();
```

6. Sort employees by name

Scenario: Sort employee names alphabetically.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> result = names.stream().sorted().toList();
```

7. Sort numbers descending

Scenario: Sort numbers from highest to lowest.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> result = nums.stream().sorted(Comparator.reverseOrder()).toList();
```

8. Count matching records

Scenario: Count orders whose amount is above 1000.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
long count = orders.stream().filter(o -> o.getAmount() > 1000).count();
```

9. Find first matching element

Scenario: Find the first employee in the IT department.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Employee> e = employees.stream().filter(x -> "IT".equals(x.getDept())).findFirst();
```

10. Check whether any match exists

Scenario: Check if any order is cancelled.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
boolean found = orders.stream().anyMatch(o -> o.isCancelled());
```

11. Check whether all match

Scenario: Verify that all scores are at least 40.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
boolean ok = scores.stream().allMatch(s -> s >= 40);
```

12. Check whether none match

Scenario: Verify that no employee is inactive.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
boolean ok = employees.stream().noneMatch(e -> !e.isActive());
```

13. Find maximum

Scenario: Find the highest transaction amount.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Transaction> max = txns.stream().max(Comparator.comparing(Transaction::getAmount));
```

14. Find minimum

Scenario: Find the lowest transaction amount.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Transaction> min = txns.stream().min(Comparator.comparing(Transaction::getAmount));
```

15. Sum integer values

Scenario: Calculate total quantity.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
int total = items.stream().mapToInt(Item::getQuantity).sum();
```

16. Average values

Scenario: Calculate average transaction amount.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
double avg = txns.stream().mapToDouble(Transaction::getAmount).average().orElse(0);
```

17. Create comma-separated string

Scenario: Join customer names using commas.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
String s = names.stream().collect(Collectors.joining(", "));
```

18. Convert list to map

Scenario: Map employee ID to employee object.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<Long, Employee> m = employees.stream().collect(Collectors.toMap(Employee::getId, Function.identity()));
```

19. Handle duplicate map keys

Scenario: Map department to employee count when duplicate keys occur.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String, Long> m = employees.stream().collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));
```

20. Group by department

Scenario: Create department -> employees map.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String, List<Employee>> m = employees.stream().collect(Collectors.groupingBy(Employee::getDept));
```

21. Partition by salary

Scenario: Split employees into salary ≥ 100000 and below.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<Boolean, List<Employee>> m = employees.stream().collect(Collectors.partitioningBy(e -> e.getSalary() >= 100000));
```

22. Group and sum

Scenario: Calculate total salary by department.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String, Double> m = employees.stream().collect(Collectors.groupingBy(Employee::getDept, Collectors.summingDouble(Employee::getSalary)));
```

23. Group and average

Scenario: Calculate average salary by department.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String, Double> m = employees.stream().collect(Collectors.groupingBy(Employee::getDept, Collectors.averagingDouble(Employee::getSalary)));
```

24. Top 3 salaries

Scenario: Return the three highest salaries.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Employee> top = employees.stream().sorted(Comparator.comparingDouble(Employee::getSalary).reversed()).limit(3).toList();
```

25. Second highest salary

Scenario: Find the second distinct highest salary.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Double> second = employees.stream().map(Employee::getSalary).distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst();
```

26. Nth highest salary

Scenario: Find the Nth distinct highest salary.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Double> nth = employees.stream().map(Employee::getSalary).distinct().sorted(Comparator.reverseOrder()).skip(n-1).findFirst();
```

27. Find duplicate values

Scenario: Find duplicate employee emails.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Set<String> seen = new HashSet<>();  
List<String> dup = emails.stream().filter(e -> !seen.add(e)).distinct().toList();
```

28. Frequency of words

Scenario: Create word -> occurrence count.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String,Long> freq = words.stream().collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

29. Most frequent word

Scenario: Find the most frequently occurring word.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Map.Entry<String,Long>> max = freq.entrySet().stream().max(Map.Entry.comparingByValue());
```

30. First non-repeated character

Scenario: Find the first character appearing once in a string.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Character> c = s.chars().mapToObj(x -> (char)x)  
    .filter(ch -> s.indexOf(ch) == s.lastIndexOf(ch)).findFirst();
```

31. First repeated character

Scenario: Find the first character that repeats.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Set<Character> seen = new HashSet<>();
Optional<Character> c = s.chars().mapToObj(x -> (char)x).filter(ch -> !seen.add(ch)).findFirst();
```

32. Sort map by value

Scenario: Sort entries by value descending.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Map.Entry<String,Integer>> out = map.entrySet().stream()
    .sorted(Map.Entry.<String,Integer>comparingByValue().reversed()).toList();
```

33. Flatten nested lists

Scenario: Convert List<List> to List.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> flat = nested.stream().flatMap(Collection::stream).toList();
```

34. Flatten sentences to words

Scenario: Convert sentences into a unique word list.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> words = sentences.stream().flatMap(x -> Arrays.stream(x.split("\\s+")))
    .distinct().toList();
```

35. Filter nulls

Scenario: Remove null customer names.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> out = names.stream().filter(Objects::nonNull).toList();
```

36. Default missing value

Scenario: Replace null names with "UNKNOWN".

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> out = names.stream().map(x -> x == null ? "UNKNOWN" : x).toList();
```

37. Convert Optional safely

Scenario: Get employee name or a default.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
String name = Optional.ofNullable(emp).map(Employee::getName).orElse("UNKNOWN");
```

38. Stream from array

Scenario: Process an array with a stream.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> out = Arrays.stream(arr).boxed().filter(n -> n > 0).toList();
```

39. Primitive stream

Scenario: Calculate sum of squares efficiently.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
int sum = nums.stream().mapToInt(Integer::intValue).map(n -> n*n).sum();
```

40. Reduce to product

Scenario: Multiply all numbers together.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
int product = nums.stream().reduce(1, (a,b) -> a*b);
```

41. Reduce to maximum

Scenario: Find max using reduce.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Integer> max = nums.stream().reduce(Integer::max);
```

42. Custom reduction

Scenario: Concatenate names with a delimiter using reduce.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
String result = names.stream().reduce("", (a,b) -> a.isEmpty() ? b : a + "," + b);
```

43. Immutable result

Scenario: Collect a stream into an unmodifiable list.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> out = names.stream().toList();
```

44. Preserve insertion order

Scenario: Remove duplicates without changing order.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> out = names.stream().distinct().toList();
```

45. Sequential vs parallel

Scenario: Parallelize CPU-heavy processing.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
long count = nums.parallelStream().filter(this::heavyCheck).count();
```

46. Avoid parallel side effects

Scenario: Explain why adding to a shared ArrayList in parallel is unsafe.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> out = nums.parallelStream().filter(this::ok).toList(); // prefer collectors/results
```

47. Custom comparator

Scenario: Sort employees by department then salary descending.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
employees.stream().sorted(Comparator.comparing(Employee::getDept)
    .thenComparing(Employee::getSalary, Comparator.reverseOrder())).toList();
```

48. Case-insensitive sort

Scenario: Sort names ignoring case.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> out = names.stream().sorted(String.CASE_INSENSITIVE_ORDER).toList();
```

49. Sort by multiple fields

Scenario: Sort by age ascending, then name.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
people.stream().sorted(Comparator.comparingInt(Person::getAge)
    .thenComparing(Person::getName)).toList();
```

50. Get youngest employee

Scenario: Find employee with minimum age.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Employee> e = employees.stream().min(Comparator.comparingInt(Employee::getAge));
```

51. Get oldest employee

Scenario: Find employee with maximum age.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Employee> e = employees.stream().max(Comparator.comparingInt(Employee::getAge));
```

52. Employees hired after date

Scenario: Filter employees hired after a given LocalDate.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
employees.stream().filter(e -> e.getJoinDate().isAfter(cutoff)).toList();
```

53. Orders in amount range

Scenario: Find orders between 500 and 5000 inclusive.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
orders.stream().filter(o -> o.getAmount() >= 500 && o.getAmount() <= 5000).toList();
```

54. Distinct departments

Scenario: Return all unique departments.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> depts = employees.stream().map(Employee::getDept).distinct().toList();
```

55. Department with most employees

Scenario: Find the department having the largest employee count.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
employees.stream().collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()))  
    .entrySet().stream().max(Map.Entry.comparingByValue());
```

56. Highest paid per department

Scenario: Find the highest-paid employee in every department.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String,Optional<Employee>> m = employees.stream().collect(Collectors.groupingBy(Employee::getDept,
    Collectors.maxBy(Comparator.comparing(Employee::getSalary))));
```

57. Lowest paid per department

Scenario: Find the lowest-paid employee in every department.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String,Optional<Employee>> m = employees.stream().collect(Collectors.groupingBy(Employee::getDept,
    Collectors.minBy(Comparator.comparing(Employee::getSalary))));
```

58. Summarize statistics

Scenario: Get count, sum, min, max and average.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
DoubleSummaryStatistics st = nums.stream().collect(Collectors.summarizingDouble(Integer::doubleValue));
```

59. Collect into LinkedHashMap

Scenario: Keep map insertion order.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<Long,Employee> m = employees.stream().collect(Collectors.toMap(Employee::getId, Function.identity(),
    (a,b) -> a, LinkedHashMap::new));
```

60. Collect into TreeSet

Scenario: Create naturally sorted unique values.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Set<String> out = names.stream().collect(Collectors.toCollection(TreeSet::new));
```

61. Group with downstream mapping

Scenario: Department -> set of employee names.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<String,Set<String>> m = employees.stream().collect(Collectors.groupingBy(Employee::getDept,
    Collectors.mapping(Employee::getName, Collectors.toSet())));
```

62. Partition and count

Scenario: Count passing and failing scores.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<Boolean,Long> m = scores.stream().collect(Collectors.partitioningBy(s -> s >= 40, Collectors.counting())
);
```

63. Joining with prefix/suffix

Scenario: Create a bracketed list of names.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
String s = names.stream().collect(Collectors.joining(", ", "[", "]"));
```

64. Use peek for debugging

Scenario: Inspect values without changing the pipeline.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> out = nums.stream().filter(n -> n > 10).peek(System.out::println).toList();
```

65. Lazy evaluation

Scenario: Explain why map/filter may not execute until a terminal operation.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
nums.stream().filter(n -> { System.out.println(n); return n > 10; }).count();
```

66. Short-circuiting

Scenario: Stop after finding a matching value.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<Integer> x = nums.stream().filter(n -> n > 100).findFirst();
```

67. Limit results

Scenario: Return only first five records.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Order> first5 = orders.stream().limit(5).toList();
```

68. Skip pagination

Scenario: Return page 3 with page size 20.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Order> page = orders.stream().skip(40).limit(20).toList();
```

69. Map object to DTO

Scenario: Convert entities to lightweight DTOs.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<EmployeeDto> out = employees.stream().map(e -> new EmployeeDto(e.getId(), e.getName())).toList();
```

70. Filter by multiple conditions

Scenario: Find active IT employees earning above a threshold.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
employees.stream().filter(Employee::isActive)
    .filter(e -> "IT".equals(e.getDept()) && e.getSalary() > 100000).toList();
```

71. Null-safe nested property

Scenario: Filter orders whose customer email exists.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
orders.stream().filter(o -> o.getCustomer() != null && o.getCustomer().getEmail() != null).toList();
```

72. Unique by property

Scenario: Keep one employee per email.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Employee> unique = employees.stream().collect(Collectors.toMap(Employee::getEmail,  
    Function.identity(), (a,b) -> a)).values().stream().toList();
```

73. Find longest string

Scenario: Find the longest name.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<String> x = names.stream().max(Comparator.comparingInt(String::length));
```

74. Find shortest string

Scenario: Find the shortest name.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Optional<String> x = names.stream().min(Comparator.comparingInt(String::length));
```

75. Total characters

Scenario: Calculate total characters across all names.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
int total = names.stream().mapToInt(String::length).sum();
```

76. Average name length

Scenario: Calculate average name length.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
double avg = names.stream().mapToInt(String::length).average().orElse(0);
```

77. Anagram check

Scenario: Check if two strings contain the same characters.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
boolean same = a.chars().sorted().boxed().toList().equals(b.chars().sorted().boxed().toList());
```

78. Character frequency

Scenario: Build character -> count map.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Map<Character, Long> m = s.chars().mapToObj(c -> (char)c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

79. Palindrome check

Scenario: Use streams to compare a string with its reverse.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
String r = new StringBuilder(s).reverse().toString();
boolean palindrome = s.equals(r);
```

80. Intersection of two lists

Scenario: Return common distinct values.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> common = a.stream().filter(new HashSet<>(b)::contains).distinct().toList();
```

81. Union of two lists

Scenario: Return distinct values from both lists.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> union = Stream.concat(a.stream(), b.stream()).distinct().toList();
```

82. Difference of lists

Scenario: Return values present in A but not B.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Set<Integer> bs = new HashSet<>(b);  
List<Integer> diff = a.stream().filter(x -> !bs.contains(x)).toList();
```

83. Top N using limit

Scenario: Find top N scores.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> top = scores.stream().sorted(Comparator.reverseOrder()).limit(n).toList();
```

84. Check sortedness

Scenario: Verify list is non-decreasing.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
boolean sorted = IntStream.range(1, nums.size()).allMatch(i -> nums.get(i-1) <= nums.get(i));
```

85. Generate sequence

Scenario: Generate first 10 integers starting at 1.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> x = IntStream.rangeClosed(1, 10).boxed().toList();
```

86. Generate random values

Scenario: Generate ten random integers from 1 to 100.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<Integer> x = new Random().ints(10, 1, 101).boxed().toList();
```

87. Date range processing

Scenario: Create dates between start and end.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<LocalDate> dates = start.datesUntil(end.plusDays(1)).toList();
```

88. Function interface: Predicate

Scenario: Create reusable validation for positive numbers.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Predicate<Integer> positive = n -> n > 0;  
List<Integer> out = nums.stream().filter(positive).toList();
```

89. Function interface: Function

Scenario: Create reusable function to calculate salary with bonus.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Function<Employee, Double> totalPay = e -> e.getSalary() + e.getBonus();  
List<Double> pay = employees.stream().map(totalPay).toList();
```

90. Function interface: Consumer

Scenario: Create reusable action to print an employee.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Consumer<Employee> printer = e -> System.out.println(e.getName());  
employees.forEach(printer);
```

91. Function interface: Supplier

Scenario: Provide a default employee lazily.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Supplier<Employee> supplier = () -> new Employee(0, "DEFAULT");  
Employee e = supplier.get();
```

92. Function interface: UnaryOperator

Scenario: Increase a number by 10.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
UnaryOperator<Integer> add10 = n -> n + 10;  
List<Integer> out = nums.stream().map(add10).toList();
```

93. Function interface: BinaryOperator

Scenario: Find the larger of two values.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
BinaryOperator<Integer> max = Integer::max;  
int result = nums.stream().reduce(max).orElse(0);
```

94. Function composition

Scenario: Compose trim and uppercase operations.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Function<String,String> clean = ((Function<String,String>)String::trim).andThen(String::toUpperCase);  
List<String> out = names.stream().map(clean).toList();
```

95. Predicate composition

Scenario: Find active employees in IT or HR.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
Predicate<Employee> active = Employee::isActive;  
Predicate<Employee> dept = e -> Set.of("IT", "HR").contains(e.getDept());  
List<Employee> out = employees.stream().filter(active.and(dept)).toList();
```

96. Custom functional interface

Scenario: Create a reusable rule for employee eligibility.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
@FunctionalInterface interface Rule<T> { boolean test(T value); }  
Rule<Employee> eligible = e -> e.isActive() && e.getSalary() > 80000;  
long count = employees.stream().filter(eligible::test).count();
```

97. Method reference

Scenario: Replace a lambda with a method reference.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> out = names.stream().map(String::toUpperCase).toList();
```

98. Constructor reference

Scenario: Create DTOs using a constructor reference.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<EmployeeDto> out = employees.stream().map(EmployeeDto::new).toList();
```

99. Optional + stream

Scenario: Process an optional employee as a stream.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
long count = optionalEmployee.stream().filter(Employee::isActive).count();
```

100. Nested optional flattening

Scenario: Flatten Optional values from a list of employees.

Solution: Apply the stream operation directly and keep lambdas stateless/side-effect free where practical.

```
List<String> emails = employees.stream().map(Employee::getOptionalEmail)  
    .flatMap(Optional::stream).toList();
```

Functional Interface Quick Reference

Predicate — `boolean test(T)` — Filtering and validation

Function — `R apply(T)` — Mapping/transformation

Consumer — `void accept(T)` — Action/side effect

Supplier — `T get()` — Lazy/default value

UnaryOperator — `T apply(T)` — Same-type transformation

BinaryOperator — `T apply(T, T)` — Combining two same-type values

BiFunction — `R apply(T, U)` — Two-input transformation

BiPredicate — `boolean test(T, U)` — Two-input validation

BiConsumer — `void accept(T, U)` — Two-input action

Senior interview discussion: lazy evaluation, short-circuiting, stateless vs stateful operations, encounter order, parallel-stream trade-offs, collector characteristics, side effects, boxing/unboxing, Optional, stream reuse, exception handling inside lambdas, and when a traditional loop is clearer than a stream.